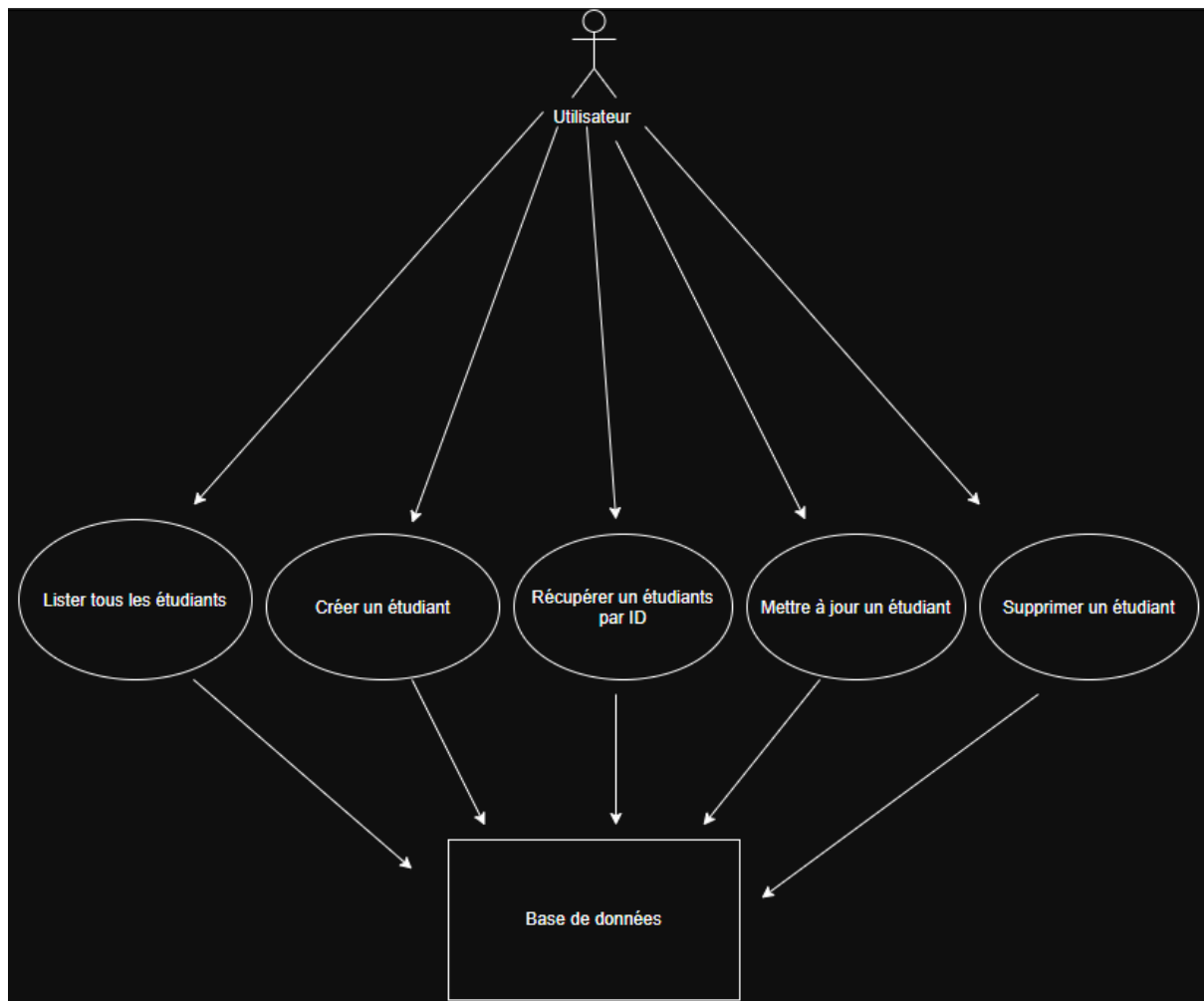
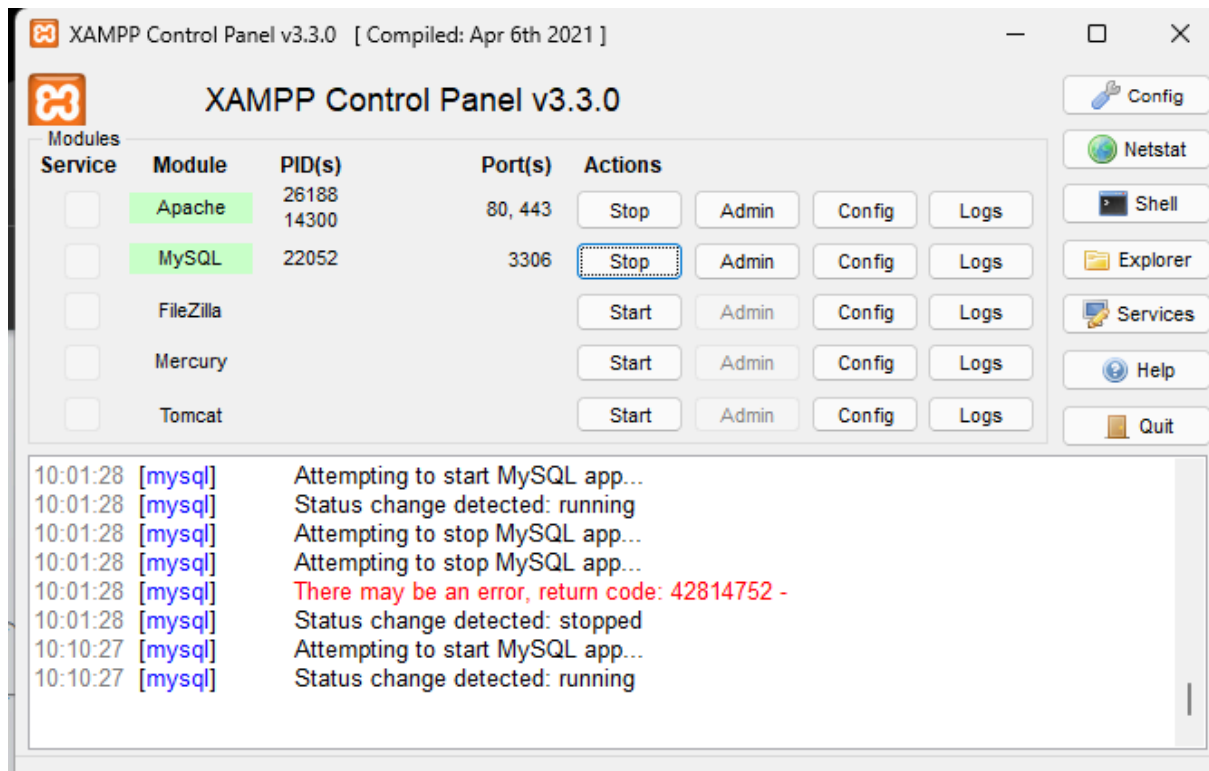


Rapport : Création d'une API REST

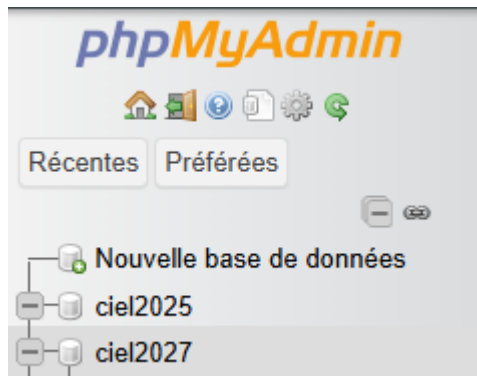
Objectif : Développer la partie “Back-end” d’une application en ligne permettant de gérer une liste d’étudiants.



Tout d’abord on doit lancer Apache et MySQL :



Ensuite on doit avoir sur phpmyadmin une base de donnée "ciel2027" :



Il faut qu'on est une table "étudiant"

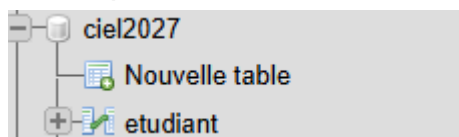


Table	Action	Lignes	Type	Interclassement	Taille	Perte
etudiant	Parcourir Structure Rechercher Insérer Vider Supprimer	11	InnoDB	utf8mb4_general_ci	16,0 kio	-

à l'intérieur :

←T→		idetudiant	nom	prenom	email	telephone
☐	Éditer Copier Supprimer	1	ALEXANDRE	Thomas	thomas.alexandre97@gmail.com	+596696123456
☐	Éditer Copier Supprimer	2	BOUTRIN	Mélissa	melissa.boutrin@gmail.com	+596696234567
☐	Éditer Copier Supprimer	3	CHALONCYS	Stéphane	stephane.chaloncys@gmail.com	+596696345678
☐	Éditer Copier Supprimer	4	DUMONT	Aurélié	aurelie.dumont972@gmail.com	+596696456789
☐	Éditer Copier Supprimer	5	EUSTACHE	Yann	yann.eustache97@gmail.com	+596696567890
☐	Éditer Copier Supprimer	18	TERIEUR	Alex		
☐	Éditer Copier Supprimer	19	TERIEUR	Alex		
☐	Éditer Copier Supprimer	22	TERIEUR	Alex		
☐	Éditer Copier Supprimer	23	TERIEUR	Alex		
☐	Éditer Copier Supprimer	25	TERIEUR	Alex		
☐	Éditer Copier Supprimer	26	Dupont	Jean		

Ensuite on doit coder une application en langage Python pour accéder à ses données

Soit la première version “api_v1”

Requête pour voir tous les étudiants de la table			
Méthode HTTP	Route	Fonction associée	Paramètres
GET	/v1/etudiants/	getEtudiants()	N/A
Réponses possibles			
Code HTTP	Données / Message en format JSON		
200	Tableau contenant tous les étudiants		
400	Requête invalide		
401	Accès non autorisé		
500	Echec de connexion à la base de données		
Requête pour voir un seul étudiant			
Méthode HTTP	Route	Fonction associée	Paramètres
GET	/v1/etudiants/{id}	getEtudiant(id)	id de l'étudiant <u>dans la route</u>
Réponse			
Code HTTP	Données / Message en format JSON		
200	Détails de l'étudiant en format JSON		
400	Requête invalide		
401	Accès non autorisé		
404	id invalide		
500	Echec de connexion à la base de données		
Requête pour ajouter un étudiant			
Méthode HTTP	Route	Fonction associée	Paramètres
POST	/v1/etudiants/	addEtudiant()	Détails de l'étudiant en format JSON <u>dans le body</u>
Réponse			
Code HTTP	Données / Message en format JSON		
201	Ajout OK		
401	Accès non autorisé		
500	Echec de connexion à la base de données		

Requête pour modifier les détails d'un étudiant			
Méthode HTTP	Route	Fonction associée	Paramètres
PUT	/v1/etudiants/{id}	updateEtudiant(id)	id de l'étudiant <u>dans la route</u> Détails de l'étudiant en format JSON <u>dans le body</u>
Réponse			
Code HTTP	Données / Message en format JSON		
200 401 404 500	Modification OK Accès non autorisé id invalide Échec de connexion à la base de données		

Requête pour supprimer un étudiant			
Méthode HTTP	Route	Fonction associée	Paramètres
DELETE	/v1/etudiants/{id}	deleteEtudiant(id)	id de l'étudiant <u>dans la route</u>
Réponse			
Code HTTP	Données / Message en format JSON		
200 401 404 500	Suppression OK Accès non autorisé id invalide Échec de connexion à la base de données		

```

from flask import Flask, jsonify, request
from db import Database
import mysql.connector
app = Flask(__name__)

mydb= mysql.connector.connect(
    host= "127.0.0.1",
    user= "root",
    password= "",
    database= "ciel2027",
    port = 3306
)
cursor = mydb.cursor()

# Récupérer tous les étudiants
@app.route('/v1/etudiants/', methods=['GET'])
def getEtudiants():

    etudiants = []

    request = "SELECT * FROM etudiant"
    cursor.execute(request)
    result = cursor.fetchall()

    for row in result:
        etudiant = {

```

```

        "idetudiant": row[0],
        "nom": row[1],
        "prenom": row[2],
        "email": row[3],
        "telephone": row[4]
    }

    etudiants.append(etudiant)

    return jsonify(etudiants), 201

# Récupérer un étudiant grâce à son ID
@app.route('/v1/etudiants/<id>', methods=['GET'])
def getEtudiant(id):
    req = "SELECT * FROM etudiant WHERE idetudiant = " + id
    print(req)

    cursor.execute(req)
    row = cursor.fetchone()
    etudiant = {
        "idetudiant": row[0],
        "nom": row[1],
        "prenom": row[2],
        "email": row[3],
        "telephone": row[4]
    }
    return jsonify(etudiant), 200

# Ajouter un étudiant
@app.route('/v1/etudiants/', methods=['POST'])
def addEtudiant():
    nom = request.json['nom']
    prenom = request.json['prenom']
    email = request.json['email']
    telephone = request.json['telephone']
    req = f"INSERT INTO etudiant (nom, prenom, email, telephone) \
        VALUES ('{nom}', '{prenom}', '{email}', '{telephone}')"
    cursor.execute(req)

```

```

mydb.commit()
return jsonify({"message": "Ajout OK"}), 201

# Mettre à jour un étudiant
@app.route('/v1/etudiants/<int:id>', methods=['PUT'])
def updateEtudiant(id):

    nom = request.json['nom']
    prenom = request.json['prenom']
    email = request.json['email']
    telephone = request.json['telephone']
    req = f"UPDATE etudiant \
        SET nom='{nom}', prenom='{prenom}', email='{email}', \
telephone='{telephone}' \
        WHERE idetudiant={id}"
    cursor.execute(req)
    mydb.commit()
    return jsonify({"message": "Mise à jour OK"}), 200

# Supprimer un étudiant
@app.route('/v1/etudiants/<int:id>', methods=['DELETE'])
def deleteEtudiant(id):

    req = f"DELETE FROM etudiant WHERE idetudiant={id}"
    cursor.execute(req)
    mydb.commit()
    return jsonify({"message": "Suppression OK"}), 200

if __name__ == '__main__':
    app.run(debug=True)

```

On a les 4 méthodes Http pour les 5 cas d'utilisations

Maintenant l'api_v2 :

```
from flask import Flask, jsonify, request
from db import Database
import mysql.connector
app = Flask(__name__)

mydb= mysql.connector.connect(
    host= "127.0.0.1",
    user= "extern_user1",
    password= " bt5@c131972",
    database= "ciel2027",
    port = 3306
)
cursor = mydb.cursor()

# Récupérer tous les étudiants
@app.route('/v2/etudiants/', methods=['GET'])
def getEtudiants():
    try :

        etudiants = []

        request = "SELECT * FROM etudiant"
        cursor.execute(request)
        result = cursor.fetchall()

        for row in result:
            etudiant = {
                "idetudiant": row[0],
                "nom": row[1],
                "prenom": row[2],
                "email": row[3],
                "telephone": row[4]
            }

            etudiants.append(etudiant)
        return jsonify(etudiants), 201

    except:
        return jsonify({"error": "Aucun étudiant trouvé"}), 404

# Récupérer un étudiant grâce à son ID
```

```

@app.route('/v2/etudiants/<id>', methods=['GET'])
def getEtudiant(id):
    req = "SELECT * FROM etudiant WHERE idetudiant = " + id
    print(req)
    try :

        cursor.execute(req)
        row = cursor.fetchone()
        etudiant = {
            "idetudiant": row[0],
            "nom": row[1],
            "prenom": row[2],
            "email": row[3],
            "telephone": row[4]
        }
        return jsonify(etudiant), 200
    except:
        return jsonify({"error": "Étudiant non trouvé"}), 404

# Ajouter un étudiant
@app.route('/v2/etudiants/', methods=['POST'])
def addEtudiant():
    try:

        nom = request.json['nom']
        prenom = request.json['prenom']
        email = request.json['email']
        telephone = request.json['telephone']
        req = f"INSERT INTO etudiant (nom, prenom, email, telephone) \
            VALUES ('{nom}', '{prenom}', '{email}', '{telephone}')"
        cursor.execute(req)
        mydb.commit()
        return jsonify({"message": "Ajout OK"}), 201
    except:
        return jsonify({"error": "Erreur lors de l'ajout"}), 400

# Mettre à jour un étudiant
@app.route('/v2/etudiants/<int:id>', methods=['PUT'])
def updateEtudiant(id):
    try:

        nom = request.json['nom']
        prenom = request.json['prenom']

```

```

        email = request.json['email']
        telephone = request.json['telephone']
        req = f"UPDATE etudiant \
                SET nom='{nom}', prenom='{prenom}', email='{email}', \
                telephone='{telephone}' \
                WHERE idetudiant={id}"
        cursor.execute(req)
        mydb.commit()
        return jsonify({"message": "Mise à jour OK"}), 200
    except:
        return jsonify({"error": "Erreur lors de la mise à jour"}), 400

# Supprimer un étudiant
@app.route('/v2/etudiants/<int:id>', methods=['DELETE'])
def deleteEtudiant(id):
    try:
        req = f"DELETE FROM etudiant WHERE idetudiant={id}"
        cursor.execute(req)
        mydb.commit()
        return jsonify({"message": "Suppression OK"}), 200
    except:
        return jsonify({"error": "Erreur lors de la suppression"}), 400

if __name__ == '__main__':
    app.run(host='0.0.0.0', debug=True)

```

On a rajouté les try et except pour mieux gérer les erreurs

Api_v3 :

```

from flask import Flask, jsonify, request
from db import Database
import mysql.connector
app = Flask(__name__)

db=Database("192.168.1.xxx", "extern_user1", " bt5@c13l972",
"ciel2027")
mydb= mysql.connector.connect(
    host= "127.0.0.1",
    user= "extern_user1",
    password= " bt5@c13l972",
    database= "ciel2027",
    port = 3306

```

```

)
cursor = mydb.cursor()

def login():
    auth = request.authorization
    username = auth.username
    password = auth.password

    req = f"SELECT * FROM user WHERE login='{username}' AND
password='{password}'"
    cursor.execute(req)
    data = cursor.fetchone()
    if data:
        return True
    else:
        return False

# Récupérer tous les étudiants
@app.route('/v3/etudiants/', methods=['GET'])
def getEtudiants():

    if login():

        try:
            etudiants = []

            request = "SELECT * FROM etudiant"
            cursor.execute(request)
            result = cursor.fetchall()

            for row in result:
                etudiant = {
                    "idetudiant": row[0],
                    "nom": row[1],
                    "prenom": row[2],
                    "email": row[3],
                    "telephone": row[4]
                }

                etudiants.append(etudiant)

            return jsonify(etudiants), 200

```

```

        except:
            return jsonify({"error": "Aucun étudiant trouvé"}), 404

    else:
        return jsonify("Accès refusé"), 401

# Récupérer un étudiant grâce à son ID
@app.route('/v3/etudiants/<id>', methods=['GET'])
def getEtudiant(id):

    if login():

        req = "SELECT * FROM etudiant WHERE idetudiant = " + id
        print(req)

        try:
            cursor.execute(req)
            row = cursor.fetchone()

            etudiant = {
                "idetudiant": row[0],
                "nom": row[1],
                "prenom": row[2],
                "email": row[3],
                "telephone": row[4]
            }

            return jsonify(etudiant), 200

        except:
            return jsonify({"error": "Étudiant non trouvé"}), 404

    else:
        return jsonify("Accès refusé"), 401

# Ajouter un étudiant
@app.route('/v3/etudiants/', methods=['POST'])
def addEtudiant():

    if login():

```

```

        try:
            nom = request.json['nom']
            prenom = request.json['prenom']
            email = request.json['email']
            telephone = request.json['telephone']

            req = f"""INSERT INTO etudiant (nom, prenom, email,
telephone)
VALUES ('{nom}', '{prenom}', '{email}', '{telephone}')"

            cursor.execute(req)
            mydb.commit()

            return jsonify({"message": "Ajout OK"}), 201

        except:
            return jsonify({"error": "Erreur lors de l'ajout"}), 400

    else:
        return jsonify("Accès refusé"), 401

# Mettre à jour un étudiant
@app.route('/v3/etudiants/<int:id>', methods=['PUT'])
def updateEtudiant(id):

    if login():

        try:
            nom = request.json['nom']
            prenom = request.json['prenom']
            email = request.json['email']
            telephone = request.json['telephone']

            req = f"""UPDATE etudiant
SET nom='{nom}', prenom='{prenom}', email='{email}',
telephone='{telephone}'
WHERE idetudiant={id}"""

            cursor.execute(req)
            mydb.commit()

            return jsonify({"message": "Mise à jour OK"}), 200

```

```

        except:
            return jsonify({"error": "Erreur lors de la mise à jour"}),
400

        else:
            return jsonify("Accès refusé"), 401

# Supprimer un étudiant
@app.route('/v3/etudiants/<int:id>', methods=['DELETE'])
def deleteEtudiant(id):

    if login():

        try:
            req = f"DELETE FROM etudiant WHERE idetudiant={id}"

            cursor.execute(req)
            mydb.commit()

            return jsonify({"message": "Suppression OK"}), 200

        except:
            return jsonify({"error": "Erreur lors de la suppression"}),
400

        else:
            return jsonify("Accès refusé"), 401

if __name__ == '__main__':
    app.run(host='0.0.0.0', debug=True)

```

On a rajouté une fonction login et on a créé un utilisateur sur phpmyadmin user1 avec comme mot de passe 123456.

Options supplémentaires

	iduser	login	password
☐ Éditer ☐ Copier ☐ Supprimer	1	user1	123456

Aussi on utilise Postman pour tester les requêtes :

ciel2027

GET lister tous les étudiants

POST créer un étudiant

GET Récupérer un étudiants par ID

PUT Mettre à jour un étudiant

DEL Supprimer un étudiant

POST

{{base_url}} / {{api_version}} /etudiants

Docs

Params

Authorization

Headers 9

Body

Scripts

Settings

Auth type

Basic Auth

The authorization header will be automatically generated when you send the request. Learn more about [authorization](#)

Basic Auth

Username

user1

Password

123456

Body

Cookies

Headers 5

Test Results 1/1

201 Created • 7 ms • 200

JSON

Preview

Visualize

```

1  {
2  |    "message": "Ajout OK"
3  |  }
4  
```

Api_v4:

```

from flask import Flask, jsonify, request
from db import Database

app = Flask(__name__)

#db = Database("192.168.1.xxx", "extern_user", "Bt5@c131972",
"ciel2027")
db = Database("127.0.0.1", "root", "", "ciel2027")

@app.route('/v4/etudiants/', methods=['GET'])
def getEtudiants():
    code = db.login(request)
    if code == 500:

```

```

        return jsonify({'message': 'Echec de connexion à la base de
données'}), 500
    if code == 401:
        return jsonify({'message': 'Accès non autorisé'}), 401

    etudiants = []
    data = db.readAll()
    if data == 401:
        return jsonify("Requête invalide"), 400
    if data == 400:
        return jsonify("Requête invalide"), 400
    for row in data:
        etudiant = {
            "idetudiant": row[0],
            "nom": row[1],
            "prenom": row[2],
            "email": row[3],
            "telephone": row[4]
        }
        etudiants.append(etudiant)
    return jsonify(etudiants), 200

@app.route('/v4/etudiants/<int:id>', methods=['GET'])
def getEtudiant(id):
    code = db.login(request)
    if code == 500:
        return jsonify({'message': 'Echec de connexion à la base de
données'}), 500
    if code == 401:
        return jsonify({'message': 'Accès non autorisé'}), 401

    data = db.readOne(id)
    if data == 400:
        return jsonify("Requête invalide"), 400
    if data != 404:
        etudiant = {
            "idetudiant": data[0],
            "nom": data[1],
            "prenom": data[2],
            "email": data[3],
            "telephone": data[4]
        }
        return jsonify(etudiant), 200

```

```

else:
    return jsonify("id invalide"), 404

@app.route('/v4/etudiants/', methods=['POST'])
def addEtudiant():
    code = db.login(request)
    if code == 500:
        return jsonify({'message': 'Echec de connexion à la base de données'}), 500
    if code == 401:
        return jsonify({'message': 'Accès non autorisé'}), 401

    data = request.get_json()
    nom = data.get('nom')
    prenom = data.get('prenom')
    email = data.get('email')
    telephone = data.get('telephone')

    result = db.create(nom, prenom, email, telephone)

    if result == 201:
        return jsonify("Etudiant ajouté avec succès"), 201
    else:
        return jsonify("Erreur lors de l'ajout de l'étudiant"), 400

@app.route('/v4/etudiants/<int:id>', methods=['DELETE'])
def deleteEtudiant(id):
    code = db.login(request)

    if code == 500:
        return jsonify({'message': 'Echec de connexion à la base de données'}), 500

    if code == 401:
        return jsonify({'message': 'Accès non autorisé'}), 401

    result = db.delete(id)

    if result == 200:
        return jsonify("Etudiant supprimé avec succès"), 200
    else:
        return jsonify("Erreur lors de la suppression de l'étudiant"),
400

```

```

@app.route('/v4/etudiants/<int:id>', methods=['PUT'])
def updateEtudiant(id):
    code = db.login(request)
    if code == 500:
        return jsonify({'message': 'Echec de connexion à la base de données'}), 500
    if code == 401:
        return jsonify({'message': 'Accès non autorisé'}), 401

    data = request.get_json()
    nom = data.get('nom')
    prenom = data.get('prenom')
    email = data.get('email')
    telephone = data.get('telephone')

    result = db.update(id, nom, prenom, email, telephone)

    if result == 200:
        return jsonify("Etudiant mis à jour avec succès"), 200
    else:
        return jsonify("Erreur lors de la mise à jour de l'étudiant"), 400

if __name__ == '__main__':
    app.run(host='0.0.0.0', port = 5000, debug=True)

```

J'ai transférer les fonctions de requête dans une classe dans une fichier db :

```

import mysql.connector
import hashlib

class Database:

    # Constructeur
    def __init__(self, host, user, password, database):
        self.host = host
        self.user = user
        self.password = password
        self.database = database

```

```

# Connexion à la base de données
def connect(self):
    connector = mysql.connector.connect(
        host = self.host,
        user = self.user,
        password = self.password,
        database = self.database)
    return connector

# Voir tous les étudiants
def readAll(self):
    connector = self.connect()
    cursor = connector.cursor()
    try:
        cursor.execute(f"SELECT * FROM etudiant")
        data = cursor.fetchall()
        return data
    except:
        return 400
    finally:
        connector.close()

# Voir un étudiant
def readOne(self, id):
    connector = self.connect()
    cursor = connector.cursor()
    try:
        cursor.execute(f"SELECT * FROM etudiant WHERE idEtudiant = {id}")
        data = cursor.fetchone()
        if data:
            return data
        else:
            return 404
    except:
        return 400
    finally:
        connector.close()

# Créer un nouvel étudiant
def create(self, nom, prenom, email, telephone):
    connector = self.connect()
    cursor = connector.cursor()

```

```

        try:
            cursor.execute(f"INSERT INTO etudiant (nom, prenom, email,
telephone) VALUES ('{nom}', '{prenom}', '{email}', '{telephone}')"
            connector.commit()
            return 201
        except:
            return 400
        finally:
            connector.close()

# Modifier un étudiant
def update(self, id, nom, prenom, email, telephone):
    connector = self.connect()
    cursor = connector.cursor()
    try:
        cursor.execute(f"UPDATE etudiant SET nom = '{nom}', prenom
= '{prenom}', email = '{email}', telephone = '{telephone}' WHERE
idEtudiant = {id}")
        connector.commit()
        return 200
    except:
        return 400
    finally:
        connector.close()

# Supprimer un étudiant
def delete(self, id):
    connector = self.connect()
    cursor = connector.cursor()
    try:
        cursor.execute(f"DELETE FROM etudiant WHERE idEtudiant =
{id}")
        connector.commit()
        return 200
    except:
        return 400
    finally:
        connector.close()

# Vérifier que les login / password existent dans la table user
def login(self, request):
    try:
        auth = request.authorization

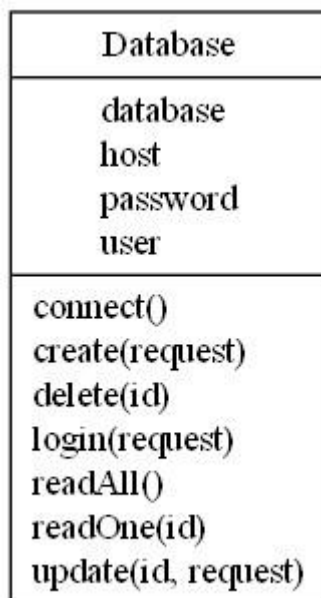
```

```

        username = auth.username
        password = auth.password
    except:
        return 401
    try:
        conn = self.connect()
        curs = conn.cursor()
    except:
        return 500
    try:
        curs.execute(f"SELECT * FROM user WHERE login = '{username}' AND password = '{password}'")
        data = curs.fetchone()
        if data:
            return 200
        else:
            return 401
    except:
        return(401)
    finally:
        conn.close()

```

Un diagramme de classes pour la classe Database généré par pyreverse :



une fiche de test de l'API sur postman de la v3 :

GET lister tous les étudiants

http://127.0.0.1:5000/v3/etudiants

200 • 7 ms • 1.178 KB 2

PASS La requête a été coorectement exécutée

PASS La requête est bien un tableau JSON

POST créer un étudiant

http://127.0.0.1:5000/v3/etudiants

201 • 10 ms • 198 B 1

PASS L'étudiant est créé (Statut 201)

GET Récupérer un étudiants par ID

http://127.0.0.1:5000/v3/etudiants/20

200 • 5 ms • 262 B 2

PASS La requête a été coorectement exécutée

PASS La requête est bien un tableau JSON

PUT Mettre à jour un étudiant

http://127.0.0.1:5000/v3/etudiants/20

200 • 8 ms • 204 B 1

PASS L'étudiant est créé (Statut 200)

DELETE Supprimer un étudiant

http://127.0.0.1:5000/v3/etudiants/20

200 • 8 ms • 199 B 1

PASS L'étudiant est créé (Statut 200)

une fiche de test de l'API sur postman de la v4 :

Iteration 1

GET lister tous les étudiants

http://127.0.0.1:5000/v4/etudiants

200 • 191 ms • 1.736 KB 2

PASS La requête a été coorectement exécutée

PASS La requête est bien un tableau JSON

POST créer un étudiant

http://127.0.0.1:5000/v4/etudiants

201 • 14 ms • 210 B 1

PASS L'étudiant est créé (Statut 201)

GET Récupérer un étudiants par ID

http://127.0.0.1:5000/v4/etudiants/23

200 • 13 ms • 262 B 2

PASS La requête a été coorectement exécutée

PASS La requête est bien un tableau JSON

PUT Mettre à jour un étudiant

http://127.0.0.1:5000/v4/etudiants/23

200 • 10 ms • 209 B 1

PASS L'étudiant est créé (Statut 200)

DELETE Supprimer un étudiant

http://127.0.0.1:5000/v4/etudiants/23

200 • 15 ms • 207 B 1

PASS L'étudiant est créé (Statut 200)